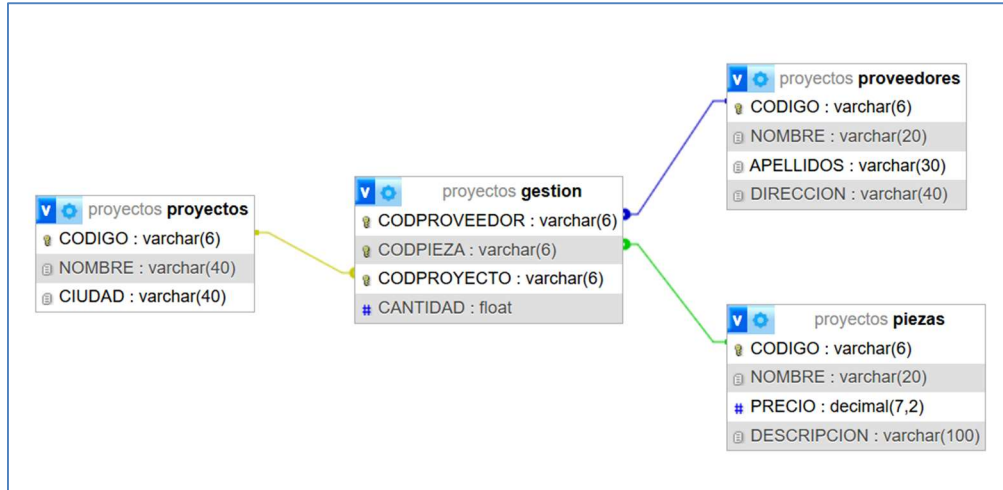


Relación de ejercicios I – BADA – Ud.6 PL-SQL

A partir de la base de datos **proyectos.sql** que puedes observar en la imagen, desarrolla las siguientes peticiones en PLSQL.



- Insertar un nuevo proyecto**
 - Escribe un procedimiento almacenado que reciba el código, nombre y ciudad de un proyecto y lo inserte en la tabla *proyectos*.
- Actualizar la dirección de un proveedor**
 - Crea un procedimiento almacenado que reciba el código de un proveedor y una nueva dirección, y actualice este dato en la tabla *proveedores*.
- Eliminar un proyecto**
 - Implementa un procedimiento almacenado que elimine un proyecto dado su código, asegurando primero que no tenga registros en la tabla *gestion*.
- Listar proveedores de una pieza**
 - Crea un procedimiento almacenado que reciba el código de una pieza y devuelva un cursor con los proveedores que suministran dicha pieza.
- Calcular el costo total de piezas en un proyecto**
 - Escribe una función que reciba el código de un proyecto y calcule el costo total de las piezas utilizadas multiplicando la cantidad por el precio de cada pieza.
- Disparador para evitar precios negativos**
 - Implementa un trigger en la tabla *piezas* que impida la inserción o actualización de precios negativos.
- Generar un informe de gestión**
 - Escribe un procedimiento almacenado que genere un informe con el número total de piezas suministradas por cada proveedor.
- Incrementar precios de piezas en un porcentaje**
 - Crea un procedimiento almacenado que reciba un porcentaje y actualice el precio de todas las piezas incrementándolo en dicho porcentaje.
- Registrar un nuevo suministro**
 - Escribe un procedimiento almacenado que agregue un nuevo registro a la tabla *gestion*, validando que los códigos de proveedor, pieza y proyecto existan.
- Disparador para evitar que se elimine un proveedor con piezas suministradas**
 - Implementa un trigger que impida la eliminación de un proveedor si tiene registros en la tabla *gestion*.

SOLUCIÓN

-- 1. Insertar un nuevo proyecto

```
CREATE OR REPLACE PROCEDURE sp_insertar_proyecto (  
    p_codigo proyectos.codigo%TYPE,  
    p_nombre proyectos.nombre%TYPE,  
    p_ciudad proyectos.ciudad%TYPE  
) AS  
  
BEGIN  
    INSERT INTO proyectos (codigo, nombre, ciudad)  
    VALUES (p_codigo, p_nombre, p_ciudad);  
    COMMIT;  
END;  
/
```

-- Bloque anónimo para probar el procedimiento

```
DECLARE  
    v_codigo proyectos.codigo%TYPE := &Codigo;  
    v_nombre proyectos.nombre%TYPE := &Nombre;  
    v_ciudad proyectos.ciudad%TYPE := &Ciudad;  
BEGIN  
    sp_insertar_proyecto(v_codigo,v_nombre,v_ciudad);  
END;
```

-- 2. Actualizar la dirección de un proveedor

```
CREATE OR REPLACE PROCEDURE sp_actualizar_direccion_proveedor (  
    p_codigo proveedores.codigo%TYPE,  
    p_nueva_direccion proveedores.direccion%TYPE  
) AS  
  
BEGIN  
    UPDATE proveedores  
    SET direccion = p_nueva_direccion  
    WHERE codigo = p_codigo;  
    COMMIT;  
END;  
/
```

-- Bloque anónimo para probar el procedimiento

```
DECLARE  
    v_codigo proveedores.codigo%TYPE := &Codigo;  
    v_direccion proveedores.direccion%TYPE := &Direccion;  
BEGIN  
    sp_actualizar_direccion_proveedor(v_codigo,v_direccion);  
END;
```

-- 3. Eliminar un proyecto

```
CREATE OR REPLACE PROCEDURE sp_eliminar_proyecto (  
    p_codigo proyectos.codigo%TYPE  
) AS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count FROM gestion WHERE codproyecto = p_codigo;
```

```
IF v_count = 0 THEN
    DELETE FROM proyectos WHERE codigo = p_codigo;
    COMMIT;
ELSE
    RAISE_APPLICATION_ERROR(-20001, 'No se puede eliminar, existen registros en gestion.');
```

END IF;

```
END;
/

-- Bloque anónimo para probar el procedimiento
DECLARE
    v_codigo proyectos.codigo%TYPE := &Codigo;
BEGIN
    sp_eliminar_proyecto(v_codigo);
END;
```

-- 4. Listar proveedores de una pieza

```
DECLARE

CURSOR c_proveedores_pieza (p_codpieza piezas.codigo%TYPE) IS
SELECT DISTINCT
    p.* FROM proveedores p
    JOIN gestion g ON p.codigo=g.codproveedor
    WHERE g.codpieza=p_codpieza;
codigo_pieza gestion.codpieza%TYPE := &cod_pieza;

BEGIN

    DBMS_OUTPUT.PUT_LINE('Proveedores de la pieza: ' || codigo_pieza);
    FOR registro IN c_proveedores_pieza(codigo_pieza) LOOP
        DBMS_OUTPUT.PUT_LINE('--Código Proveedor: ' || registro.codigo);
        DBMS_OUTPUT.PUT_LINE('--Nombre Proveedor: ' || registro.nombre);
        DBMS_OUTPUT.PUT_LINE('--Apellidos Proveedor: ' || registro.apellidos);
        DBMS_OUTPUT.PUT_LINE('--Dirección Proveedor: ' || registro.direccion);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

END;
/
```

-- 5. Calcular el costo total de piezas en un proyecto

```
CREATE OR REPLACE FUNCTION fn_calcular_costo_proyecto (
    p_codproyecto proyectos.codigo%TYPE
) RETURN NUMBER AS
    v_total NUMBER := 0;
BEGIN
    SELECT SUM(g.cantidad * pi.precio) INTO v_total
    FROM gestion g
    JOIN piezas pi ON g.codpieza = pi.codigo
    WHERE g.codproyecto = p_codproyecto;
    RETURN NVL(v_total, 0);
END;
/
```

```
-- Bloque anónimo para probar el procedimiento
DECLARE
  v_codigo proyectos.codigo%TYPE := &Codigo;
BEGIN
  DBMS_OUTPUT.PUT_LINE('El costo del proyecto '
    || v_codigo || ' es: '
    || fn_calcular_costo_proyecto(v_codigo)
  );
END;
```

-- 6. Disparador para evitar precios negativos

```
CREATE OR REPLACE TRIGGER validar_precio_pieza
BEFORE INSERT OR UPDATE ON piezas
FOR EACH ROW
BEGIN
  IF :NEW.precio < 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'El precio no puede ser negativo.');
```

END IF;

```
END;
/

-- Bloque anónimo para la prueba del procedimiento
DECLARE

BEGIN
  INSERT INTO piezas VALUES ('PEO', 'PRODUCTO ERR', -100, 'PRUEBA DE PIEZA ERRÓNEA');
```

END;

```
/
```

-- 7. Generar un informe de gestión

```
DECLARE

CURSOR c_piezas_proveedor IS
SELECT
  p.codigo,
  p.apellidos,
  p.nombre,
  SUM(g.cantidad) AS total_piezas
FROM proveedores p
JOIN gestion g ON p.codigo = g.codproveedor
GROUP BY p.codigo, p.apellidos, p.nombre;

BEGIN

  DBMS_OUTPUT.PUT_LINE('Total de piezas suministradas por proveedores: ');
  FOR registro IN c_piezas_proveedor LOOP
    DBMS_OUTPUT.PUT_LINE('--Código Proveedor: ' || registro.codigo);
    DBMS_OUTPUT.PUT_LINE('--Nombre Proveedor: ' || registro.nombre);
    DBMS_OUTPUT.PUT_LINE('--Apellidos Proveedor: ' || registro.apellidos);
    DBMS_OUTPUT.PUT_LINE('--TOTAL PIEZAS: ' || registro.total_piezas);
    DBMS_OUTPUT.PUT_LINE('-----');
```

END LOOP;

```
END;
```

/

-- 8. Incrementar precios de piezas en un porcentaje

```
CREATE OR REPLACE PROCEDURE sp_incrementar_precio_piezas (  
    p_porcentaje NUMBER  
) AS  
BEGIN  
    UPDATE piezas  
    SET precio = precio * (1 + p_porcentaje / 100);  
    -- COMMIT;  
END;  
/  
  
-- Bloque anónimo para la prueba del procedimiento  
DECLARE  
  
BEGIN  
    sp_incrementar_precio_piezas(10);  
END;  
/
```

-- 9. Registrar un nuevo suministro

```
CREATE OR REPLACE PROCEDURE sp_registrar_suministro (  
    p_codproveedor gestion.codproveedor%TYPE,  
    p_codpieza gestion.codpieza%TYPE,  
    p_codproyecto gestion.codproyecto%TYPE,  
    p_cantidad gestion.cantidad%TYPE  
) AS  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count FROM proveedores WHERE codigo = p_codproveedor;  
    IF v_count = 0 THEN RAISE_APPLICATION_ERROR(-20003, 'Proveedor no existe.');
```

END IF;

```
    SELECT COUNT(*) INTO v_count FROM piezas WHERE codigo = p_codpieza;  
    IF v_count = 0 THEN RAISE_APPLICATION_ERROR(-20004, 'Pieza no existe.');
```

END IF;

```
    SELECT COUNT(*) INTO v_count FROM proyectos WHERE codigo = p_codproyecto;  
    IF v_count = 0 THEN RAISE_APPLICATION_ERROR(-20005, 'Proyecto no existe.');
```

END IF;

```
    INSERT INTO gestion (codproveedor, codpieza, codproyecto, cantidad)  
    VALUES (p_codproveedor, p_codpieza, p_codproyecto, p_cantidad);  
END;  
/  
  
-- Bloque anónimo para la prueba del procedimiento  
DECLARE  
  
BEGIN  
    sp_registrar_suministro('PR0002', 'L0002', 'ED0001', 2);  
END;  
/
```

-- 10. Disparador para evitar eliminar un proveedor con suministros

```
CREATE OR REPLACE TRIGGER evitar_eliminar_proveedor
```

BEFORE DELETE ON proveedores FOR EACH ROW

DECLARE

v_count NUMBER;

BEGIN

SELECT COUNT(*) INTO v_count FROM gestion g WHERE g.codproveedor= :OLD.codigo;

IF v_count > 0 THEN

RAISE_APPLICATION_ERROR(-20006, 'No se puede eliminar proveedor con suministros registrados.');

END IF;

END;

/

-- Bloque anónimo para la prueba del procedimiento

DECLARE

BEGIN

DELETE FROM proveedores WHERE codigo='PR0005';

END;

/

